

Guia Integral de la Entrega 3 del Proyecto Fin de Curso

Capa de datos distribuida y pipeline analítico paralelo

Aplicaciones Distribuidas · Séptimo semestre · ISR-701

Universidad Técnica Estatal de Quevedo · Facultad de Ciencias de la Computación

Carrera: Ingeniería de Software

Asignatura: Aplicaciones Distribuidas (ISR-701)

Documento: Guía PFC Entrega 3 (E3)

Semana de entrega: Semana 13

Período: Regular 2026-2027

Modalidad: Grupal (equipos PFC de 3-4 integrantes)

Docente responsable: Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Versión: v1.0

1. Presentación y lógica acumulativa de la entrega

La Entrega 3 (E3) del Proyecto Fin de Curso (PFC) no es un ejercicio aislado, sino la continuación natural del sistema distribuido que cada equipo viene construyendo desde la Entrega 1 (E1) y la Entrega 2 (E2). En E1 los equipos fundamentaron el problema, propusieron la arquitectura y prototiparon la comunicación entre procesos con sockets y gRPC bajo el modelo lógico de Lamport, con lo que quedaron establecidos el dominio, los actores, los requisitos no funcionales y el estilo arquitectónico. En E2 los equipos materializaron ese diseño como una arquitectura de microservicios REST contenerizada con Docker Compose, con API Gateway y autenticación basada en *JSON Web Tokens* (JWT) según la especificación RFC 7519 [1, 2, 3].

La E3 completa esa arquitectura con las dos capas que faltan para que el sistema sea *distribuido en sentido pleno*: (i) una capa de persistencia distribuida real, con fragmentación, replicación y consenso, sustituyendo las bases embebidas o mono-nodo utilizadas en E2; y (ii) un pipeline de procesamiento paralelo que consume esos datos, mide y demuestra empíricamente la aceleración (*speedup*) obtenida. Con estos dos añadidos la aplicación queda con arquitectura distribuida completa a nivel de servicios, datos y cómputo, condición necesaria para que la Entrega 4 (E4) aborde en el siguiente ciclo la calidad, la interfaz web, la interfaz móvil y la observabilidad de manera integral.

Principio de complementariedad E3 → E4. Todo artefacto producido en E3 debe ser reutilizado, ampliado o instrumentado en E4. En consecuencia, esta guía excluye deliberadamente actividades desechables y prioriza artefactos duraderos: esquema de BDD, cluster de CockroachDB, pipeline PySpark y protocolo experimental replicable.

2. Objetivos

2.1. Objetivo general

Consolidar la capa de datos distribuida y el procesamiento paralelo del sistema del PFC, integrándolos con la arquitectura de microservicios de E2, y evaluar experimentalmente su rendimiento y tolerancia a fallos con el rigor metodológico propio de la experimentación en ingeniería de software [4].

2.2. Objetivos específicos

1. Diseñar el esquema distribuido del sistema con estrategias explícitas de fragmentación (horizontal o mixta) y replicación, fundamentadas en el modelo de acceso del dominio [5, 6].
2. Desplegar un cluster CockroachDB de tres nodos en contenedores y refactorizar el microservicio principal de E2 para que consuma ese cluster en lugar de la base de datos mono-nodo original [7].

3. Verificar de manera reproducible las propiedades de consistencia serializable y de tolerancia a fallos mediante caída controlada de nodos, apoyándose en el algoritmo de consenso Raft [8].
4. Ejecutar un protocolo experimental de comparacion de rendimiento entre el cluster distribuido y una instancia mono-nodo equivalente, reportando media, desviación típica e intervalo de confianza al 95 % [9, 10].
5. Implementar un pipeline analitico en Apache Spark (PySpark) que consume datos del cluster, aplica al menos cinco transformaciones y expone su rendimiento contra un baseline secuencial en `pandas` [11, 12, 13].
6. Comprobar experimentalmente la Ley de Amdahl [14] y comparar los resultados con la reevaluacion propuesta por Gustafson [15].
7. Documentar la totalidad del trabajo en un manuscrito LaTeX con estructura afin al reporte de un experimento controlado, siguiendo las guias metodológicas de Wohlin et al. [4] y Runeson y Höst [16].

3. Aplicación por equipo PFC

La Tabla 1 concreta la aplicación de cada bloque de la E3 sobre los cuatro equipos del período, tomando como base el dominio establecido en E1 y los microservicios implementados en E2.

Tabla 1: Aplicación concreta de E3 por equipo PFC (arrastré desde E1–E2 + adición propia de E3).

Equipo	Fragmentación + colocación	Consultas de referencia	Analítica (Spark) paralela
AGLS TiendaTech	Fragmentación horizontal de pedidos por <code>fecha_pedido</code> (partition by range trimestral); colocación clientes $\bowtie$ pedidos por <code>cliente_id</code> .	Top-N productos por trimestre; embudo de conversion; <i>cross-sell</i> entre categorías de hardware.	Recomendador colaborativo con <code>spark.ml</code> sobre historial simulado ($\geq 500k$ pedidos); segmentacion RFM.
BCEL SGA Escuela	Fragmentación horizontal de calificaciones por <code>período_academico</code> ; colocación con <code>matriculas</code> por <code>estudiante_id</code> .	Reporte de rendimiento por asignatura y período; detección de estudiantes en riesgo; series temporales por curso.	Modelado predictivo de aprobacion/reprobacion; análisis de correlacion asistencia–rendimiento sobre $\geq 500k$ registros sintéticos.
FUVV Laboratorios	Fragmentación horizontal de reservas por <code>fecha</code> ; particionado por sala en tablas de monitoreo.	Uso por franja horaria; picos por laboratorio; correlacion incidentes–carga.	Análisis de series temporales de telemetría; detección de anomalías con IQR y ventanas móviles.
ACC Soporte ISP	Fragmentación horizontal de tickets por <code>fecha_apertura</code> ; colocación clientes $\bowtie$ tickets por <code>cliente_id</code> .	SLA por tipo de incidente; tiempo medio de resolucion; escalamientos por zona geográfica.	Análisis de reincidencia; agrupamiento (clustering) de incidentes por texto y zona.

4. Fundamento teórico exigible

Este bloque delimita el cuerpo teórico mínimo que el documento LaTeX del equipo debe desarrollar. Se exige rigor conceptual: definiciones formales, notacion consistente, discusión de *trade-offs* y citas primarias.

4.1. Modelo de bases de datos distribuidas

Un sistema de base de datos distribuido (SBDD) se define como un conjunto de múltiples bases de datos lógicamente relacionadas, gestionadas por un único sistema y percibidas por el usuario como una única base [5]. Sus objetivos son transparencia (fragmentación, replicación y ubicación), autonomía local, disponibilidad y escalabilidad horizontal.

La *fragmentación horizontal* de una relación R produce fragmentos $R_1, R_2, \dots, R_n$ mediante selección sobre un atributo particionador: $R_i = \sigma_{p_i}(R)$, siendo $\{p_i\}$ predicados mutuamente excluyentes y colectivamente exhaustivos. La *replicación* exige un factor $f \geq 3$ para tolerar la caída de un nodo con quórum mayoritario. Un sistema con N nodos y factor de replicación f tolera la pérdida simultánea de hasta $\lfloor (f-1)/2 \rfloor$ réplicas por rango, siempre que se mantenga el quórum de escritura $\lceil f/2 \rceil + 1$ [8, 7].

4.2. Teorema CAP y modelo PACELC

El teorema CAP establece que un sistema distribuido no puede satisfacer simultáneamente consistencia, disponibilidad y tolerancia a particiones [17]. El modelo PACELC extiende esta discusión incorporando el trade-off entre latencia y consistencia en ausencia de particiones [18]. CockroachDB se ubica como sistema CP con consistencia serializable, priorizando corrección sobre disponibilidad ante particiones [7].

4.3. Consenso distribuido: Raft

El algoritmo Raft descompone el consenso en tres subproblemas: elección de líder, replicación de logs y seguridad [8]. Cada rango de datos en CockroachDB constituye un grupo Raft independiente con su propio líder. Una escritura se considera comprometida cuando el líder recibe confirmación de una mayoría estricta de réplicas, garantizando durabilidad frente a la caída de una minoría.

4.4. Modelo de programación paralela

El modelo RDD (*Resilient Distributed Dataset*) de Spark abstrae una colección distribuida inmutable con linaje de transformaciones, evaluación perezosa y tolerancia a fallos mediante recomputo [11]. La Ley de Amdahl expresa el speedup teórico máximo alcanzable:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}} \quad (1)$$

donde p es la fracción paralelizable del programa y N el número de procesadores. La reevaluación de Gustafson replantea el enfoque asumiendo que el volumen de trabajo escala con los recursos disponibles [15].

5. Manual de desarrollo (paso a paso)

Este bloque es el núcleo técnico de la guía. Cada módulo describe entradas, procedimiento, salidas esperadas y criterios de aceptación. Se asume que el equipo ya cuenta con la solución de E2 en su rama `main` y que crea una rama `feature/entrega-3` para el trabajo de esta entrega.

5.1. Stack tecnológico consolidado

Stack obligatorio para E3

- **Persistencia distribuida:** CockroachDB v23.2+, cluster de 3 nodos en Docker.
- **Backend:** el mismo microservicio principal del PFC (Java 21 + Spring Boot 3.x o equivalente ya adoptado en E2), refactorizado para usar el driver JDBC PostgreSQL apuntando al cluster.
- **Contenerización:** Docker $\geq$ 24, Docker Compose plugin v2.
- **Analítica paralela:** Apache Spark 3.5+ (PySpark) sobre Python 3.11+.
- **Instrumentación de métricas:** Prometheus client library (heredado de E2), extendida con contadores y *histogramas* nuevos para las consultas distribuidas.
- **Documentación y CI/CD:** el repositorio, el pipeline GitHub Actions y la documentación LaTeX se heredan de E2 y solo se amplían.

5.2. Módulo A: refactorización del microservicio de E2

Entrada: microservicio principal de E2 (por ejemplo, `resource-service`) con persistencia mononodo (H2, SQLite o PostgreSQL local).

Procedimiento:

1. Crear una rama `feature/entrega-3` desde `main`.
2. Sustituir la dependencia embebida por el driver PostgreSQL 42.x (compatible con el protocolo CockroachDB).
3. Externalizar la cadena de conexión vía variables de entorno (`DB_URL`, `DB_USER`, `DB_PASS`); documentar en `.env.example`.
4. Introducir un *profile* `crdb` en la configuración de Spring Boot (o equivalente) que activa la conexión al cluster.
5. Adaptar los repositorios: usar SQL portable, evitar sentencias específicas de un motor y explicitar transacciones con nivel `SERIALIZABLE`.
6. Actualizar las pruebas de integración para levantar el cluster con `Testcontainers` y no depender de una base local.

Salida esperada: el microservicio compila, arranca y responde igual que en E2, pero contra el cluster CockroachDB. Todas las pruebas de E2 siguen en verde.

5.3. Módulo B: diseño del esquema distribuido

Entrada: modelo entidad-relación consolidado en E1.

Procedimiento:

1. Identificar la tabla *de mayor cardinalidad* y su patrón de acceso predominante (rango temporal, geográfico, categorial).
2. Definir el esquema de fragmentación horizontal con cláusulas `PARTITION BY RANGE` o `PARTITION BY LIST` de CockroachDB [19].
3. Establecer colocación con `INTERLEAVE IN PARENT` o mediante índices geo-particionados cuando corresponda al dominio.
4. Fijar el factor de replicación en 3 (por defecto) mediante `ALTER RANGE default CONFIGURE ZONE USING num_replicas = 3`.
5. Documentar la decisión en un *Architectural Decision Record* (ADR) siguiendo la plantilla de Nygard, almacenado en `docs/adr/ADR-003-fragmentación.md`.

Salida esperada: script `db/schema.sql` versionado, ADR firmado y diagrama del esquema en `docs/diagrams/db-schema.drawio.png`.

5.4. Módulo C: cluster CockroachDB con Docker Compose

Procedimiento: añadir al `docker-compose.yml` de E2 tres nodos CockroachDB y un servicio de inicialización, tal como se muestra en el Listado 1.

```

1 services:
2   crdb-1:
3     image: cockroachdb/cockroach:v23.2.4
4     command: start --insecure --advertise-addr=crdb-1
5               --join=crdb-1,crdb-2,crdb-3
6               --http-addr=0.0.0.0:8081
7     ports: ["26257:26257", "8081:8081"]
8     volumes: ["crdb1-data:/cockroach/cockroach-data"]
9     networks: [pfc-net]
10
11   crdb-2:
12     image: cockroachdb/cockroach:v23.2.4
13     command: start --insecure --advertise-addr=crdb-2
14               --join=crdb-1,crdb-2,crdb-3
15     volumes: ["crdb2-data:/cockroach/cockroach-data"]
16     networks: [pfc-net]
17
18   crdb-3:
19     image: cockroachdb/cockroach:v23.2.4
20     command: start --insecure --advertise-addr=crdb-3
21               --join=crdb-1,crdb-2,crdb-3
22     volumes: ["crdb3-data:/cockroach/cockroach-data"]
23     networks: [pfc-net]
24
25   crdb-init:
26     image: cockroachdb/cockroach:v23.2.4
27     depends_on: [crdb-1, crdb-2, crdb-3]
28     entrypoint: /bin/sh
29     command: -c "sleep 5 && cockroach init
30               --insecure --host=crdb-1
31               && cockroach sql --insecure --host=crdb-1
32               -f /docker-entrypoint-initdb.d/schema.sql"
33     volumes: ["/db:/docker-entrypoint-initdb.d"]
34     networks: [pfc-net]
35
36 volumes:
37   crdb1-data: {}
38   crdb2-data: {}
39   crdb3-data: {}
40
41 networks:
42   pfc-net: {}
43

```

Listing 1: Extensión del `docker-compose.yml` de E2 con el cluster CockroachDB.

Verificación: desde el nodo líder ejecutar `cockroach node status --insecure` y confirmar que los tres nodos aparecen como `is_live=true` y `is_available=true`.

5.5. Módulo D: verificación experimental de tolerancia a fallos

Este módulo produce evidencia audiovisual y numérica del comportamiento Raft.

1. Con `docker compose up -d` levantar el sistema completo.
2. Poblar la tabla de mayor cardinalidad con al menos 10^5 filas usando un generador reproducible.
3. Grabar en vídeo: (a) consulta `SELECT COUNT(*)` exitosa; (b) parada abrupta de un nodo con `docker kill crdb-2`; (c) reconsulta que debe seguir siendo exitosa; (d) reincorporación con `docker start crdb-2` y espera de la reconvergencia.
4. Registrar en la tabla de resultados la latencia observada antes, durante y después del fallo.
5. Ejecutar la misma prueba deteniendo dos nodos: se espera pérdida de disponibilidad (com-

portamiento CP consistente con [7]).

Salida esperada: vídeo en docs/evidencias/tolerancia_fallos.mp4 y bitacora docs/evidencias/tolerancia_fallos.md.

5.6. Módulo E: pipeline analítico paralelo

Procedimiento:

1. Crear spark/ en el repositorio con un script pipeline.py (PySpark) y un notebook análisis.ipynb equivalente.
2. Extraer datos del cluster mediante el conector JDBC PostgreSQL configurando particiones de lectura por rango.
3. Aplicar **exactamente** cinco transformaciones: filtrado, joins entre tablas colocalizadas, agregación con ventanas, transformación de tipos temporales y una operación de ml (StringIndexer, Bucketizer o similar).
4. Persistir la salida en formato Parquet en spark/out/.
5. Implementar el *baseline secuencial* equivalente con pandas en spark/baseline.py.

El Listado 2 muestra el esqueleto mínimo del pipeline.

```

1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3 from pyspark.sql.window import Window
4 import time
5
6 def run(url, user, password, table, k):
7     spark = (SparkSession.builder
8             .appName("pfc-e3-pipeline")
9             .master(f"local[{k}]")
10            .config("spark.sql.shuffle.partitions", 8)
11            .getOrCreate())
12     t0 = time.perf_counter()
13     df = (spark.read.format("jdbc")
14          .option("url", url).option("dbtable", table)
15          .option("user", user).option("password", password)
16          .option("driver", "org.postgresql.Driver")
17          .option("partitionColumn", "id")
18          .option("lowerBound", 1).option("upperBound", 5_000_000)
19          .option("numPartitions", 16).load())
20     t1 = df.filter(F.col("estado") == "OK")
21     t2 = t1.groupBy("categoria").agg(F.count("*").alias("n"),
22                                     F.avg("monto").alias("prom"))
23     w = Window.partitionBy("categoria").orderBy(F.desc("prom"))
24     t3 = t2.withColumn("rk", F.row_number().over(w))
25     t4 = t3.withColumn("dia", F.to_date("fecha"))
26     t5 = t4.filter(F.col("rk") <= 10)
27     t5.write.mode("overwrite").parquet("out/topN")
28     t_end = time.perf_counter() - t0
29     print(f"cores={k}  t_total_s={t_end:.3f}")
30     spark.stop()
31     return t_end
32

```

Listing 2: Esqueleto de spark/pipeline.py.

5.7. Módulo F: protocolo experimental (rigor JCR)

Para que los resultados sean defendibles como evidencia empírica, el equipo debe seguir el siguiente protocolo:

1. **Hipótesis operativa:** H_1 el pipeline paralelo con N cores presenta menor tiempo medio de ejecución que el baseline secuencial para $|D| \geq 5 \cdot 10^5$ registros; H_0 no hay diferencia.
2. **Variables:** independiente $N \in \{1, 2, 4, 8\}$; dependientes tiempo total t , tiempo por transfor-

mación t_i , uso de CPU y memoria residente.

3. **Diseño:** bloque completo con $r = 10$ repeticiones por nivel de N ; se descartan la primera y última repetición (calentamiento y enfriamiento).
4. **Análisis:** media $\bar{t}$, desviación típica s , intervalo de confianza al 95 % $\bar{t} \pm 1.96 \cdot s / \sqrt{r}$; contraste con prueba t pareada si se cumple normalidad, Wilcoxon en caso contrario.
5. **Amenazas a la validez:** listar como mínimo tres amenazas internas (calentamiento JVM, ruido del sistema anfitrión, caché OS) y dos externas (representatividad del dataset, generalidad a otras cargas).

Este protocolo sigue las recomendaciones de Jain [9] y Wohlin et al. [4] y se documenta en `docs/experimentos/protocolo.md`.

5.8. Módulo G: instrumentación de métricas incrementales

Extender el endpoint `/metrics` del microservicio principal (heredado de E2) con nuevas métricas Prometheus:

- `crdb_query_duration_seconds` (*histogram*, buckets 5 ms–2 s).
- `crdb_transaction_retries_total` (*counter*).
- `crdb_pool_active_connections` (*gauge*).

Estas métricas se emplearán tal cual en la E4 para construir el dashboard consolidado; por eso se instrumentan aquí y no después.

6. Documento LaTeX exigido

El manuscrito de E3 se estructura como un *reporte experimental*, con las secciones mínimas de la Tabla 2. El texto debe tener al menos 15 páginas de contenido (sin portada ni bibliografía) y como mínimo 10 referencias, de las cuales al menos seis provienen de revistas o conferencias indexadas (IEEE Xplore, ACM DL, Springer, Elsevier). El estilo bibliográfico es IEEE (biblatex `IEEEtran` o bibtex `ieeetr`).

Tabla 2: Estructura mínima del manuscrito LaTeX de la Entrega 3.

Sección	Contenido	Páginas ref.
Resumen	Español + inglés, ≤ 200 palabras cada uno, estructurado objetivo-método-resultado-conclusión.	1
Introducción	Contexto acumulativo (que se hereda de E1–E2), problema abordado en E3, contribuciones.	1–1.5
Trabajos relacionados	Al menos 5 fuentes indexadas sobre BDD, CockroachDB, Spark, Amdahl.	1–1.5
Fundamento teórico	Los cuatro bloques de la Sección 4 con notación y citas.	2–2.5
Diseño del esquema distribuido	Fragmentación, replicación, ADR, diagrama del esquema.	1.5–2
Cluster y su verificación	Módulo C y D, con capturas y bitácora del vídeo.	1.5–2
Pipeline paralelo	Módulo E; código en <code>lstlisting</code> ; explicación de cada transformación.	2–2.5

Sección	Contenido	Páginas ref.
Protocolo y resultados	Módulo F; tabla de mediciones, ecuación (1), IC 95 %.	2.5–3
Discusión	Interpretación, amenazas a la validez, limitaciones.	1–1.5
Conclusiones e integración con E4	Que artefactos de E3 se reutilizan en E4.	0.5–1
Bibliografía	IEEE, ≥ 10 referencias, la mayoría con DOI resoluble.	—

7. Estructura del repositorio

El repositorio de E3 es la **misma** instancia iniciada en E1 y ampliada en E2; se crea la rama `feature/entrega-3` y, al final, se fusiona a `main` por *pull request* con revisión cruzada entre integrantes. El Listado 3 muestra la estructura resultante después del *merge* de esta entrega.

```

1 proyecto-pfc/
2   README.md           # actualizado con sección "Entrega 3"
3   LICENSE
4   .env.example        # incluye credenciales del cluster
5   docker-compose.yml  # ampliado con cluster CockroachDB
6   docs/
7     adr/
8       ADR-001-arquitectura.md # heredado de E1
9       ADR-002-microservicios.md # heredado de E2
10      ADR-003-fragmentación.md # nuevo
11      ADR-004-consenso-raft.md # nuevo
12   api/
13     openapi.yaml      # heredado de E2, con nuevas rutas si aplica
14   db/
15     schema.sql        # nuevo
16     seeds.sql         # nuevo
17     diagrams/
18       db-schema.drawio.png # nuevo
19   diagrams/
20     c4-nivel2.drawio.png # heredado de E1
21     c4-nivel3.drawio.png # actualizado con el cluster
22   evidencias/
23     tolerancia_fallos.mp4 # nuevo
24     tolerancia_fallos.md  # nuevo
25   experimentos/
26     protocolo.md        # nuevo
27     resultados/
28       raw.csv           # datos crudos
29       boxplot.png       # figura para el manuscrito
30   entrega1/ entrega2/ entrega3/ # snapshots de PDFs
31   services/
32     resource-service/   # microservicio principal de E2, refactorizado
33     auth-service/       # sin cambios
34     notification-service/ # sin cambios
35   spark/
36     pipeline.py         # nuevo
37     baseline.py         # nuevo
38     requirements.txt    # nuevo
39     análisis.ipynb      # nuevo
40     out/               # ignorado por git
41   tests/
42     integration/        # heredado, ahora con Testcontainers CRDB
43     load/              # heredado
44   .github/
45     workflows/
46       ci.yml           # heredado de E2, con job "crdb-tests"
47

```

Listing 3: Estructura del repositorio después de fusionar E3.

8. Rúbrica de evaluación

La rúbrica opera con siete dimensiones, dieciseis criterios y cinco niveles (0 a 4). Se aplica la fórmula estándar de la asignatura:

$$\text{Nota}_{/10} = \left[\frac{\sum_i w_i \cdot n_i}{4} \right] \cdot 10 \quad , \quad \sum_i w_i = 1.0 \quad (2)$$

donde w_i es el peso del criterio i y $n_i \in \{0, 1, 2, 3, 4\}$ es su nivel alcanzado. La ausencia de un indicador se puntúa CERO en el criterio correspondiente; esta condición es no negociable y coherente con la política de la asignatura.

Tabla 3: Rúbrica de evaluación de la Entrega 3.

Dim.	Criterio	Indicador clave nivel 4	Peso
D1	1.1 Esquema distribuido y fragmentación	Estrategia justificada por patrón de acceso, con ADR y colocalización	9 %
	1.2 Diseño de replicación y factor Raft	Factor $f = 3$, discusión de quórum, tolerancia demostrada	8 %
D2	2.1 Cluster de 3 nodos funcionando	<code>node status</code> verde, esquema aplicado, healthchecks pasan	8 %
	2.2 Verificación de tolerancia a fallos	Vídeo + bitacora, latencia registrada, prueba con 2 nodos caídos	10 %
D3	3.1 Integración microservicio → cluster	Refactor limpio, Testcontainers en verde, transacciones serializable	8 %
	3.2 Métricas Prometheus incrementales	Los 3 instrumentos exigidos exportados y validados con <code>promtool</code>	4 %
D4	4.1 Pipeline PySpark completo	Las 5 transformaciones, salida en Parquet, notebook reproducible	10 %
	4.2 Baseline pandas y comparativa	Ambos scripts, mismos datos, resultados numéricos comparables	4 %
	4.3 Protocolo experimental JCR	$r = 10$ repeticiones, IC 95 %, contraste estadístico, amenazas	8 %
D5	5.1 Estructura del manuscrito	Todas las secciones de la Tabla 2, resumen bilingüe	5 %
	5.2 Uso de LaTeX y calidad tipográfica	Compila limpio, figuras con <code>\caption/\label</code> , tablas <code>booktabs</code> , <code>listings</code>	4 %
	5.3 Bibliografía	IEEE, ≥ 10 referencias, ≥ 6 indexadas con DOI resoluble	4 %
D6	6.1 Repositorio y trazabilidad	Ramas, PR con revisión cruzada, commits equilibrados por integrante	4 %
	6.2 Reproducibilidad	<code>docker compose up</code> levanta todo, README actualizado, semillas deterministas	4 %
D7	7.1 Defensa oral (grupal)	Todos los integrantes responden preguntas técnicas sin leer	6 %
	7.2 Ética y honestidad académica	Declaración de uso de IA, atribución correcta, ausencia de plagio	4 %

9. Anexos

Anexo A. Lista de verificación final

- ☐ Rama `feature/entrega-3` fusionada a `main` por PR con revisión.
- ☐ Cluster de 3 nodos operativo con `docker compose up -d`.
- ☐ Esquema aplicado con la estrategia de fragmentación documentada en ADR.

- ☐ Vídeo de tolerancia a fallos ≤ 5 minutos, con audio explicativo.
- ☐ Pipeline PySpark y baseline pandas ejecutan sin error sobre el dataset de referencia.
- ☐ Tabla de resultados con al menos 10 repeticiones por nivel de N .
- ☐ Manuscrito LaTeX compila sin errores; ≥ 15 páginas de contenido.
- ☐ Bibliografía con ≥ 10 referencias IEEE y ≥ 6 con DOI verificado.
- ☐ Repositorio con README actualizado y `.env.example` vigente.

Anexo B. Preguntas obligatorias para la defensa

1. Explique como el algoritmo Raft garantiza consistencia serializable en su cluster ante la caída de un nodo, apoyándose en su bitacora experimental.
2. Justifique la clave de particionado elegida en su tabla principal a partir del patrón de acceso del PFC y estime el impacto en las consultas de referencia.
3. Presente la Ley de Amdahl con los valores de p estimados a partir de sus mediciones y calcule el número de cores con el cual se alcanza el 90 % del speedup teórico máximo.
4. Discuta al menos tres amenazas a la validez interna de su experimento y como las mitigo.
5. Indique que artefactos de esta entrega serán consumidos, directamente y sin modificación, por la Entrega 4.

Referencias

- [1] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT).” IETF RFC 7519, 2015.
- [2] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [3] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media, 2 ed., 2021.
- [4] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [5] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Cham, Switzerland: Springer, 4 ed., 2020.
- [6] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [7] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “CockroachDB: The resilient geo-distributed SQL database,” in *Proc. 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1493–1509, 2020.
- [8] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 305–319, 2014.
- [9] R. Jain, “The art of computer systems performance analysis: Techniques for experimental design, measurement, simulation, and modeling,” *Wiley*, 1991.
- [10] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *Proc. 1st ACM Symposium on Cloud Computing (SoCC)*, pp. 143–154, 2010.

- [11] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012.
- [12] B. Chambers and M. Zaharia, *Spark: The Definitive Guide: Big Data Processing Made Simple*. Sebastopol, CA, USA: O’Reilly Media, 2018.
- [13] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, “Big data analytics on Apache Spark,” *International Journal of Data Science and Analytics*, vol. 1, no. 3–4, pp. 145–164, 2016.
- [14] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- [15] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [16] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [17] E. Brewer, “CAP twelve years later: How the “rules” have changed,” *IEEE Computer*, vol. 45, no. 2, pp. 23–29, 2012.
- [18] D. J. Abadi, “Consistency tradeoffs in modern distributed database system design: CAP is only part of the story,” *IEEE Computer*, vol. 45, no. 2, pp. 37–42, 2012.
- [19] Cockroach Labs, “CockroachDB documentation v23.2.” Online documentation, 2024.